Aluno: Daniel Nunes Santos **Matrícula:** 22652850 **Turma:** ABI-01

RELATÓRIO TÉCNICO - INTELIGÊNCIA ARTIFICIAL

Análise de Arquitetura de Rede Neural

Resumo Executivo: Este documento apresenta a análise técnica detalhada do script Python/NumPy fornecido. Trata-se de um Perceptron Multicamadas (MLP) implementado do zero (*from scratch*), treinado com o algoritmo de Backpropagation e otimização por Descida do Gradiente em um problema de regressão multi-saída.

1. Especificações da Arquitetura

Componente	Parâmetro no Código	Dimensão / Valor	Descrição
Camada de Entrada	D_in	1.000 neurônios	Vetor de características (*features*) de entrada.
Camada Oculta	H	100 neurônios	Camada intermediária de representação.
Camada de Saída	D_out	10 neurônios	Dimensão do vetor de predições.
Tamanho do Lote	N	64 amostras	Quantidade de dados processados por iteração (Batch Size).
Termo de Viés (*Bias*)	—	Ausente	Operações baseadas estritamente em multiplicação de matrizes.

2. Funções Matemáticas e Dinâmica de Treino

- **Ativação da Camada Oculta:** Função Sigmóide definida por $h = 1 / (1 + e^{-x \cdot w1})$.
- **Ativação da Camada de Saída:** Linear (sem transformação adicional sobre $y_pred = h \cdot w2$).
- **Função de Perda (*Loss*):** Erro Quadrático Total (*Sum of Squared Errors*), calculada por $L = \sum (y_pred - y)^2$.
- **Otimizador:** Descida do Gradiente Gradual (*Vanilla Gradient Descent*) com taxa de aprendizado $\alpha = 10^{-4}$ (0.0001).
- **Ciclo de Treinamento:** Executado por 2.000 iterações/épocas com atualização manual dos pesos $w1$ e $w2$.

3. Algoritmo de Backpropagation (Diferenciação Manual)

Diferente de frameworks modernos que utilizam autograd (como PyTorch ou TensorFlow), a rede calcula os gradientes manualmente pela Regra da Cadeia:

```
grad_y_pred = 2.0 * (y_pred - y) grad_w2 = h.T.dot(grad_y_pred) grad_h =
grad_y_pred.dot(w2.T) grad_w1 = x.T.dot(grad_h * h * (1 - h))
```

4. Considerações Técnicas

- **Tipo de Tarefa:** Regressão Multi-Saída contínua. Os alvos e entradas são gerados a partir de uma distribuição normal standard.
- **Inicialização de Pesos:** Inicialização aleatória via distribuição normal padronizada (`np.random.randn`).
- **Transparência e Desempenho:** Estrutura didática e eficiente para demonstrar o funcionamento básico do algoritmo de retropropagação sem dependências externas.